==========1

(REATE FUNCTION multiple (x INT, y INT

$$ RETURNS INT AS

BEGIN

;RETURN x * y

;END

;LANGUAGE plpgsql $$

ALTER TABLE

CREATE FUNCTION

```
ON UPDATE CASCADE;CREATE FUNCTION multiple (x INT, y INT)
RETURNS INT AS $$
BEGIN
  RETURN x * y;
END;
$$ LANGUAGE plpgsql;
ALTER TABLE
CREATE FUNCTION
postgres=# select multiple (3,5)
postgres-# ;
 multiple
----------
       15
(1 row)

postgres=#
```

==========2

```
postgres=# CREATE FUNCTION hallo_world(name text)
RETURNS text AS $$
BEGIN
  RETURN 'welcome ' || name;
END;
$$ LANGUAGE plpgsql;
ERROR:  function "hallo_world" already exists with same argument types
postgres=# SELECT hallo_world('Hussein');
   hallo_world
-----------------
 welcome Hussein
(1 row)

postgres=#
```

==========3

```
postgres=# CREATE FUNCTION check_num(num int )
RETURNS text AS $$
BEGIN
   if num%2=0  then return 'even';
   else return 'odd';
    end if;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=# select check_num(7)
postgres-# ;
 check_num
-----------
 odd
(1 row)

postgres=#
```

==========4

```
CREATE   FUNCTION get_student(student_id INT)
RETURNS TABLE(id INT, name varchar(100),email varchar(150), address text, trach_id int, birth_date date, gender varchar(6) ) AS $$
BEGIN
    RETURN QUERY
    SELECT s.id, s.e_name, s.email, s.address, s.track_id , s.birth_date , s.gender
    FROM student s
    WHERE s.id = student_id;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=# SELECT get_student(1);
                    get_student
---------------------------------------------------
 (1,"Ahmed Ali",ahmed@example.com,Cairo,1,2002-02-05,)
(1 row)

postgres=#
```

==========5

```
postgres=# CREATE  FUNCTION subject_avg(sub_name VARCHAR)
RETURNS NUMERIC AS $$
DECLARE
    avg_grade NUMERIC;
BEGIN
    SELECT AVG(g.grade)
    INTO avg_grade
    FROM grades g
    JOIN subject s ON g.sub_id = s.id
    WHERE s.sub_name = sub_name;

     return avg_grade;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=# SELECT get_subject_avg('Python');
   get_subject_avg
---------------------
 90.0000000000000000
(1 row)

postgres=#
```

=======7

```
CREATE FUNCTION
postgres=# CREATE TABLE student_audit (
    id SERIAL PRIMARY KEY,
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    action_type VARCHAR(10),  -- INSERT, UPDATE, DELETE
    student_id INT,
    description TEXT
);
ERROR:  relation "student_audit" already exists
postgres=# CREATE OR REPLACE FUNCTION log_student_changes()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO student_audit (action_type, student_id, description)
        VALUES ('INSERT', NEW.id, 'Student added: ' || NEW.e_name);

    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO student_audit (action_type, student_id, description)
        VALUES ('UPDATE', NEW.id, 'Student updated: ' || NEW.e_name);

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO student_audit (action_type, student_id, description)
        VALUES ('DELETE', OLD.id, 'Student deleted: ' || OLD.e_name);
    END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=# CREATE TRIGGER trg_student_changes
AFTER INSERT OR UPDATE OR DELETE ON student
FOR EACH ROW
EXECUTE FUNCTION log_student_changes();
CREATE TRIGGER
postgres=# INSERT INTO student (e_name, email, address, track_id, birth_date, gender)
VALUES ('Ali Hassan', 'ali@example.com', 'Cairo', 1, '2000-01-01', 'Male');
```

```
ERROR:  relation "student_audit" already exists
postgres=# CREATE OR REPLACE FUNCTION log_student_changes()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO student_audit (action_type, student_id, description)
        VALUES ('INSERT', NEW.id, 'Student added: ' || NEW.e_name);

    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO student_audit (action_type, student_id, description)
        VALUES ('UPDATE', NEW.id, 'Student updated: ' || NEW.e_name);

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO student_audit (action_type, student_id, description)
        VALUES ('DELETE', OLD.id, 'Student deleted: ' || OLD.e_name);
    END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=# CREATE TRIGGER trg_student_changes
AFTER INSERT OR UPDATE OR DELETE ON student
FOR EACH ROW
EXECUTE FUNCTION log_student_changes();
CREATE TRIGGER
postgres=# INSERT INTO student (e_name, email, address, track_id, birth_date, gender)
VALUES ('Ali Hassan', 'ali@example.com', 'Cairo', 1, '2000-01-01', 'Male');
INSERT 0 1
postgres=# SELECT * FROM student_audit;
 id |        action_time         | action_type | student_id |        description
----+----------------------------+-------------+------------+---------------------------
  1 | 2025-05-01 15:40:28.266178 | INSERT      |          6 | Student added: Ali Hassan
(1 row)

postgres=#
```